Technological University of the Philippines – Manila

College of Engineering

Electronics Engineering Department

Lab5_Algorithms_Big0

1. Emergency Dispatch Record Search

Original Dataset: [25, 28, 31, 34, 37, 40, 43, 46, 49, 52]

Rotated Dataset: [40, 43, 46, 49, 52, 25, 28, 31, 34, 37]

Target Incident: 43

Rotated Binary Search Comparison Count: 2

Linear Search Comparison Count: 2

Rotated Binary Search Time Complexity: $O(\log N)$

Linear Search Time Complexity: $O(N)$

Space Complexity: $O(1)$

```
========================================
EXERCISE 1: Emergency Dispatch Record Search
========================================
Original Dataset:  [25, 28, 31, 34, 37, 40, 43, 46, 49, 52]
Rotated Dataset:   [40, 43, 46, 49, 52, 25, 28, 31, 34, 37]
Target Incident:   25
Rotated Search:    Found at index 5 with 3 comparisons
Linear Search:     Found at index 5 with 6 comparisons
Time Complexity:   O(log N) for Rotated Binary Search | O(N) for Linear Search
Space Complexity:  O(1)
```

2. University Enrollment Records

Department A Dataset: [10, 30, 50, 70, 90]

Department B Dataset: [12, 16, 20, 24, 28]

Consolidated Dataset: [10, 12, 16, 20, 24, 28, 30, 50, 70, 90]

Number of Records Before Consolidation: 10

Number of Records After Consolidation: 10

Comparison Count: 6

Ordering Verification: SUCCESS

Time Complexity: $O(N + M)$

Space Complexity: $O(N + M)$

```
========================================
EXERCISE 2: University Enrollment Records
========================================
Department A dataset:  [10, 30, 50, 70, 90]
Department B dataset:  [12, 16, 20, 24, 28]
Consolidated dataset:  [10, 12, 16, 20, 24, 28, 30, 50, 70, 90]
Records before/after:  Before = 10 | After = 10
Comparison count:      6
Ordering verification: SUCCESS
Time Complexity:       O(N + M)
Space Complexity:      O(N + M)
```

Technological University of the Philippines – Manila
College of Engineering
Electronics Engineering Department
Lab5_Algorithms_Big0

3. E-Commerce Order Processing

Generated Order Dataset: [89, 42, 94, 90, 89, 13, 13, 85, 38, 98, 87, 35, 71, 74, 80]

Sorted Order Input: [13, 13, 35, 38, 42, 71, 74, 80, 85, 87, 89, 89, 90, 94, 98]

Reverse-Ordered Input: [98, 94, 90, 89, 89, 87, 85, 80, 74, 71, 42, 38, 35, 13, 13]

Random Input: [89, 42, 94, 90, 89, 13, 13, 85, 38, 98, 87, 35, 71, 74, 80]

Bubble Sort Comparison Count (Sorted): 105

Bubble Sort Comparison Count (Reverse): 105

Bubble Sort Comparison Count (Random): 105

Selection Sort Comparison Count (Sorted): 105

Selection Sort Comparison Count (Reverse): 105

Selection Sort Comparison Count (Random): 105

Insertion Sort Comparison Count (Sorted): 14

Insertion Sort Comparison Count (Reverse): 104

Insertion Sort Comparison Count (Random): 59

Bubble Sort Movement Count (Sorted): 0

Bubble Sort Movement Count (Reverse): 312

Bubble Sort Movement Count (Random): 162

Selection Sort Movement Count (Sorted): 0

Selection Sort Movement Count (Reverse): 21

Selection Sort Movement Count (Random): 36

Insertion Sort Movement Count (Sorted): 28

Insertion Sort Movement Count (Reverse): 118

Insertion Sort Movement Count (Random): 73

Sorting Verification (All Conditions & Algorithms): SUCCESS

Bubble Sort Best-Case Time Complexity: $O(N^2)$

Bubble Sort Worst/Average-Case Time Complexity: $O(N^2)$

Selection Sort Best/Worst/Average-Case Time Complexity: $O(N^2)$

Insertion Sort Best-Case Time Complexity: $O(N)$

Insertion Sort Worst/Average-Case Time Complexity: $O(N^2)$

Best-Case Observation: Insertion Sort is the most efficient on sorted data, requiring only 14 comparisons.

Average-Case Observation: Insertion Sort performs best on random data with fewer comparisons (59) and movements (73). **Worst-Case Observation:** Selection Sort minimizes data movements (21 moves) on reverse-ordered data despite high comparisons.

